

Differentiating Malware from Cleanware Using Behavioural Analysis

Ronghua Tian, Rafiqul Islam and Lynn Batten
School of IT
Deakin University, Melbourne, Australia
{rtia,rislam,lmbatten}@deakin.edu.au

Steve Versteeg
CA Labs
Melbourne, Australia
Steve.Versteeg@ca.com

Abstract

This paper proposes a scalable approach for distinguishing malicious files from clean files by investigating the behavioural features using logs of various API calls. We also propose, as an alternative to the traditional method of manually identifying malware files, an automated classification system using runtime features of malware files. For both projects, we use an automated tool running in a virtual environment to extract API call features from executables and apply pattern recognition algorithms and statistical methods to differentiate between files. Our experimental results, based on a dataset of 1368 malware and 456 cleanware files, provide an accuracy of over 97% in distinguishing malware from cleanware. Our techniques provide a similar accuracy for classifying malware into families. In both cases, our results outperform comparable previously published techniques.

Key words: Malware, strings, API, dynamic.

1 Introduction

With the rise of the shadow Internet economy, malware is no longer just used to damage, break or intrude on computer network systems, but now exists primarily as a tool used by criminals to make profit [11]. This has driven an enormous increase in new malware appearing each day. The traditional method of manually determining a signature for each new malware file has been shown to be unscalable. Anti-malware vendors are now employing new techniques to create generic signatures capable of detecting multiple variants of malware. However there are no clean-cut rules for generically distinguishing malware from cleanware. In fact, the API calls employed by malware in their malicious behaviour have legitimate use in clean software thus making it difficult for anti-virus developers to develop generic techniques which detect malware with 100% accuracy.

An added complication is the current trend towards cloud computing. Businesses are migrating their mission critical applications to cloud computing platforms and malware writers are expected to start targeting users of cloud computing. Therefore anti-virus researchers need to update their strategy to defend against malware on cloud computing services. Cloud computing requires real-time, timely responses of the anti-malware analysis system. Furthermore, in a cloud computing environment, knowing whether a file

is malicious or clean is more urgent than knowing the specific family from which the malware is from. Given this situation, developing a rapid, automatic and efficient software differentiation system is extremely urgent.

However, the performance of such a system relies heavily on the techniques adopted. Techniques used in the research literature to detect and classify malware can be divided into two main groups: static extraction and dynamic extraction. Static extraction techniques refer to scanning the binary code to determine properties of the program without running it. These techniques were first utilised by compiler developers in code optimization and reverse engineering and later widely used in malware detection [2, 3, 8]. As malware anti-detection techniques, such as polymorphism and metamorphism, continued to improve, people realized that static extraction alone would not be sufficient to detect malicious code and that dynamic analysis is a necessary complement to static detection techniques. The reason for including dynamic analysis is that dynamic techniques can monitor the instructions that are actually executed by a program and which are immune to many of the code obfuscating transformations. Dynamic analysis monitors the execution of malware, usually in a virtualised or emulated environment. As malware is continually evolving, dynamic analysis is becoming the dominant direction of anti-malware research.

In this paper we use dynamic API call sequences as features to develop an automatic, dynamic malware classification system. In our experiments, we execute each file in a virtual machine environment for a limited time to obtain trace reports which reveal the actual behaviour of the code. We then extract features from the trace reports. These two stages of our work provide a new contribution to software extraction research. We then move to the third stage: distinguishing of cleanware from malware. Our method gives an accuracy of over 97%. Finally, in the fourth stage of the paper, we classify our malware files by family, again achieving an accuracy of over 97%. Our accuracy in both cases is better than that of any previously published generic classification techniques.

The rest of the paper is organized as follows: In Section 2 we summarize the relevant literature in this area. Section 3 describes our feature extraction method while Section 4 develops the statistical methodology and the malware versus cleanware classification. In Section 5, we present the malware family classification. Section 6

provides discussion and comparison of our method with comparable existing methods; conclusions follow in Section 7.

2 Related Work

Traditional anti-malware detection and classification systems are based on static features extracted from executables by reverse-engineering. In [10,15,16,17] the authors proposed an approach for detecting malware in the windows platform by analysing the windows API calling sequence which relates to the behaviour of a piece of PE (Portable Executable) code. This information in PE files is static and different from the true calls of program, and it is easily modified by obfuscation technology, thus is no longer reliable as a malware classifier. In [9], the authors used opcode sequence frequencies to calculate the cosine similarity between two PE executable files. These opcode sequences are also static and can be evaded by obfuscation technology.

In [6], the authors explored the limitation of static extraction methodology. In this paper, they introduced a code obfuscation schema which demonstrates that static extraction alone is not enough to either detect or classify malicious code; they propose that dynamic extraction is a necessary complement to static techniques as it is significantly less vulnerable to code obfuscating transformations.

More and more researchers are now working on dynamic techniques to improve the effectiveness and accuracy of malware extraction and classification. In [14] the authors used dynamic extraction technologies to classify malware by using a controller to manage execution, stopping the execution after 10 seconds. They firstly calculate the similarity between two API call sequences by constructing a similarity matrix based on action codes (actually the sequence of API calls).

They then compute the relative frequency of each function call and construct a second matrix based on the Hellinger distance, which measures how much information is contained in each malware file. Finally, two phylogenetic trees are constructed using the similarity matrix and the Hellinger distance matrices separately. Their test set is a small set of 104 malware files and their algorithm has relatively high time and space complexities. In addition they did not include any clean code in the test; nor did the authors mention the classification accuracy of their work.

In [1] the authors proposed a composite method which extracts statistical features from both spatial and temporal information available in run-time API calls. Spatial features are generally statistical properties such as means, variances and entropies of address pointers and size parameters; the temporal feature chosen was the n th order discrete time Markov chain [11] in which each state corresponds to a particular API call. They use 237 core API calls from six different functional categories and use a 10-fold cross validation procedure with five standard classification algorithms. Their method gives good results with 96.3% classification accuracy but at the cost of high computational complexity.

3 Extractions of Behavioural Features from API Calls

In this section, we describe the process of file execution and collection of run-time trace reports based on API system calls. First we generate a list of files from our existing database “Ida2DB” [18] which includes the module ID, family and file location. Then we execute the files one by one in our virtual environment and collect execution logs.

3.1 Files Used

The malware used in our experiment was collected from CA’s VET zoo (www.ca.com), along with a collection of our own clean files. The files from VET Zoo were already classified into families. In our tests we used files from 10 malware families with a total of 1368 variants, as shown in Table 1. The first 7 families have been pre-classified as trojans; the next 3 families as viruses. The clean files were taken from the System32 directories of Windows 98, Windows 2000, Windows 2003 Windows XP and Windows 7. We chose system files as our clean files as these are likely to make some of the same low level API calls as are used by malicious programs, thus providing a challenging task for our methodology. In all, we have used 456 cleanware files from Windows System32 directories.

Table 1: Experimental dataset

	Family	No. of Files
Malware Files classified as families	Clagger	47
	Robknot	119
	Alureon	56
	Bambo	70
	Beovens	144
	Boxed	366
	Emerleox	77
	Looked	67
	Agobot	340
	Robzip	82
	Sub Total	1368
Cleanware files	Cleanware	456
	Total	1824

3.2 Trace Tool

The trace tool, which we call HookMe, was developed by anti-malware researchers at HCL Technologies and is built around Microsoft technology called Detours [4] which is designed to intercept Win32 functions by rewriting target function images and hooking API functions that are then imported into an executable. HookMe focuses on configuring Detours to specify what to collect and then directs output to a trace report. It thus actively monitors the state changes in the system and generates a trace report which reflects the behaviours of the files in terms of API function calls.

In our testing, we recorded the runtime behaviour of binaries using the tool HookMe, which then logs various events (for example, which API calls the binary makes). In our experiments we generated trace reports for each

binary file (including cleanware and malware executables). We then extracted features from the trace reports for each file and passed them into our classification engine as described in sections 4.2 and 5.1.

The model used in our classification process is shown in Figure 1 which includes three stages: (1) data acquisition, (2) file execution and log file construction, and (3) feature extraction and classification.

3.3 Trace Report Construction

In our experiment, each file is executed and monitored using our trace tool (HookMe). We set up our experimental environment by configuring a virtual machine (VM) (hosted on VMware Server version 2.0.1) on which we installed the Windows XP operating system and disabled networking. Within this virtual machine environment, we ran HookMe to monitor the execution of each file. We terminated the execution after 30 seconds for each file and exported the trace report outside of the

virtual machine environment for further analysis. Our process is described in the following steps:

- Step 1: Save the pre-configured VM state by creating a snapshot.
- Step 2: Revert to snapshot.
- Step 3: Copy one binary file into the VM
- Step 4: Execute the file using HookMe which will generate a trace report.
- Step 5: Kill the VM.
- Step 6: Copy the trace report off the VM
- Step 7: Repeat steps 2 to 6 for all files.
- Step 8: Collate the trace reports for further analysis.

We collected all the trace reports from both malware and cleanware files and then used the trace information to extract features. A file trace report is shown in Figure 2, where each field is separated by commas.

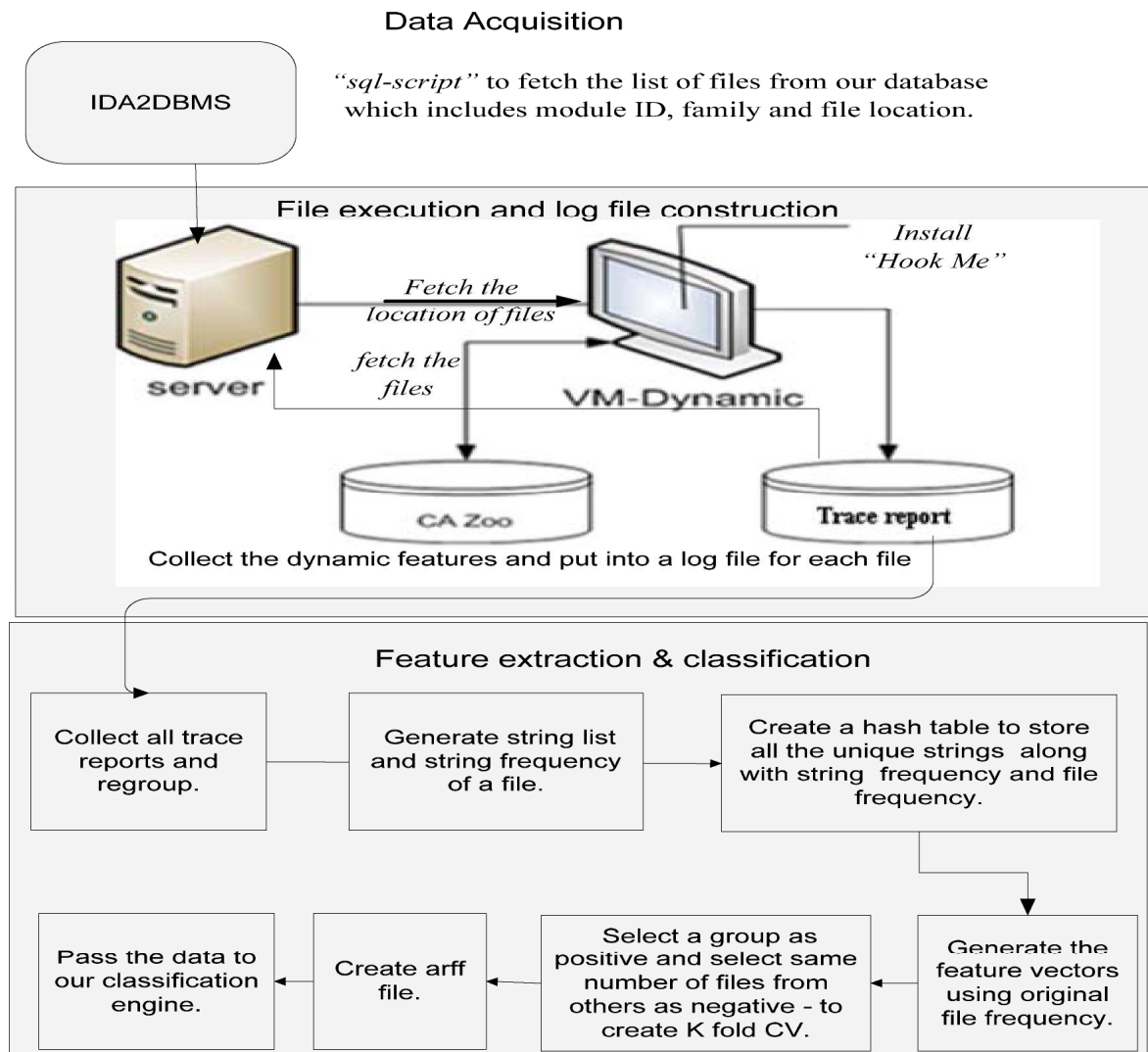


Figure 1: Our proposed classification model

1.	2009/09/07 22:01:05.249, 212, LoadLibraryExW, C:\WINDOWS\system32\untheme.dll
2.	2009/09/07 22:01:05.249, 212, RegOpenKeyExW, 0x50, Software\Microsoft\Windows\CurrentVersion\ThemeManager
3.	2009/09/07 22:01:05.249, 212, RegQueryValueExW, Composting
	009/09/07 22:01:05.249, 212, RegOpenKeyExW, 0x50, Control Panel\Desktop
4.	2009/09/07 22:01:05.249, 212, RegQueryValueExW, LaneButtonText
5.	2009/09/07 22:01:05.249, 212, LoadLibraryW, NUxTheme.dll
6.	2009/09/07 22:01:05.249, 212, LoadLibraryExW, NUxTheme.dll
7.	2009/09/07 22:01:05.249, 212, LoadLibraryExW, C:\WINDOWS\system32\untheme.dll
8.	2009/09/07 22:01:05.249, 212, LoadLibraryExW, C:\WINDOWS\system32\untheme.dll
9.	2009/09/07 22:01:05.249, 212, LoadLibraryExW, C:\WINDOWS\system32\untheme.dll
10.	2009/09/07 22:01:05.249, 212, GetProcAddress, 0x0, RegisterPenApp
11.	2009/09/07 22:01:05.249, 212, RegQueryValueExW, 1fEscapement
12.	2009/09/07 22:01:05.249, 212, RegQueryValueExW, 1fOrientation
13.	2009/09/07 22:01:05.249, 212, RegQueryValueExW, 1fWeightsuccessfully detoured
14.	2009/09/07 22:01:05.249, 216, All detouring in place

Figure 2: A file trace report of executables

4. Malware versus Cleanware Classification

In this section we introduce a method for comparing the behaviour of malicious executables versus clean files using trace reports. Our aim is for a generic method which is able to classify whether a family is malware regardless of its specific family.

4.1 Feature Extraction

In the malware versus cleanware experiment – we followed recommendations of [7] in using an equal portion of malware and of cleanware in the training as this gives the best classification results. Since we had three times as many malware files as cleanware files, we needed to run (at least) three tests in order to incorporate all the files at least once into our testing. We created groups with equal numbers of malware and cleanware files.

We then generated string lists for each group. Our feature extraction method is to separate out all the API call strings and their parameters which occur in the trace reports. We treat the API calls and their parameters as separate strings. First we read the strings from our trace reports and record each string into our global list (line numbers and dates are ignored). We use a hash table to store all strings along with their global frequency and file frequency; we call it f_s and f_m respectively. The f_s is the total number of occurrences of the string in all files and the f_m is the number of occurrences of the string within a file. We then compare the strings in each file against the global list.

For each file we calculated f_m for all the strings in the global list. We also kept track of the global frequencies. Table 2 gives an example of the global frequencies and file frequencies for a small set of strings. In our classification experiments we found that only information about the presence of a string was important, however including the number of occurrences of the string, f_m , did not improve the classification results. Therefore our final feature vectors for each file only recorded which of the strings in the global list was present in each file. In our feature vectors we used "1" to denote that a string was present and "0" to denote that a string was not present.

Features	Global fr.	f_s
P6	26	MSVCRT.dll
P7	41	DontUseDNSLoadBalancing
P8	27	WSAJoinLeaf
P16	25	AutoConfigURI.
P17	25	RtlConvertSidToUnicodeString
P18	27	CreateTimerQueue
P19	1	0x1ae3
P150	41	UpdateSecurityLevel
P151	1	Domains\gotadoronline.com
P160	27	WSAEnumNameSpaceProvidersW
P461	5	Kernel32.dll
P833	27	Ws2_32NumHandleBuckets
P834	9	\\MountPointManager

Figure 3 . Example feature sets of a malware file

The following example clarifies our feature extraction process. Let us assume that we consider three files, with the strings extracted from the execution logs of these files listed as follows:

- File 1 = {GetProcAddress, RegQueryValueExW, CreateFileW, GetProcAddress}
- File 2= {GetProcAddress, OpenFile, FindFirstFileA, FindNextFileA, CopyMemory}
- File 3 = {GetProcAddress, CreateFileW, CopyMemory}

Table 2 lists the global frequencies and the file frequencies which would be generated for the strings in this example.

Table 2: Example global frequencies and file frequencies

String list	Global frequency	File frequency
GetProcAddress	4	3
RegQueryValueExW	2	2
CreateFileW	2	2
OpenFile	1	1
FindFirstFileA	1	1
FindNextFileA	1	1
CopyMemory	2	2

The feature vectors for the example files would be :

- File 1 = {1,1,1,0,0,0,0}
- File 2 = {1,0,0,1,1,1,1}
- File 3 = {1,0,1,0,0,0,1}.

4.2 Classification Process

In our classification we integrated the Weka library (www.cs.waikato.ac.nz/ml/weka/) with our system. Weka is a set of data mining tools used mainly for classification and clustering. It is a collection of machine learning algorithms to perform data mining tasks, which includes such techniques as decision trees and tables, rule learners, Bayesian classification methods, support vector machines (SVM) and multi-layer neural networks. Algorithms can either be applied directly to a dataset or called from Java code. In all our experiments, we used k-fold cross validation [5] for classifying between malware and cleanware, with k =10, as this is a standard, well understood and well accepted method of classification in a number of domains including malware analysis.

We tested four base classifiers from Weka: SVM, Random Forest, Decision Table and the instance-based classifier, IB1. These were chosen to represent a spectrum of analysis techniques as in our previous paper [18]. In addition, the meta-classifier and booster AdaboostM1 was tested added to each of the base classifiers.

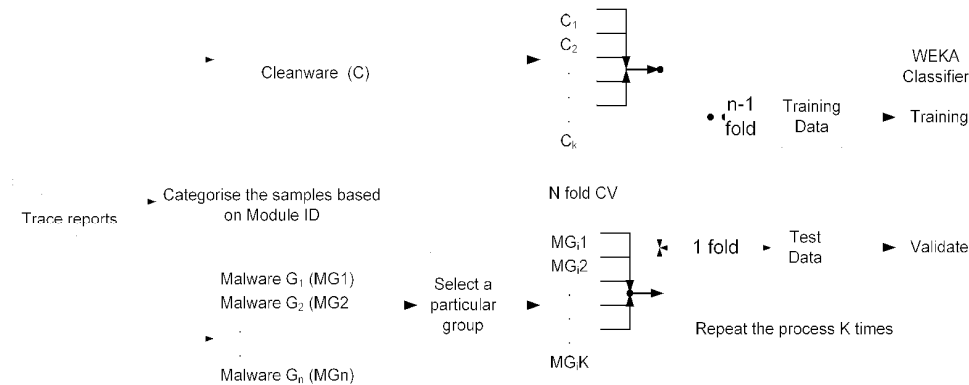


Figure 4: The k-fold validation process.

malware group. The methodology used in our experiment is given below:

- Step 1: Extract string information from the collected trace reports.
- Step 2: Divide the malware files into 3 groups with size equal to that of the cleanware set.
- Step 3: Build a hash table to record the global and file string frequencies (f_g and f_m).
- Step 4: Create feature vectors as described in section 4.1 and construct the WEKA (arff) file.
- Step 5: Select cleanware files and split them k fold.
- Step 6: Select a malware group and split it k fold
- Step 7: Build *Training Data* and *Test Data* sets.
- Step 8: Select the classifier C_i [$i = 1..n$]
- Step 9: Call WEKA libraries to train the C_i using *Training Data*
- Step 10: Evaluate the *Test Data* set.
- Step 11: Repeat until finish all N folds.
- Step 12: Repeat for other classifiers.
- Step 13: Repeat for other malware groups.

4.3 Experimental Results

Table 3 shows the average of the experimental results for each classifier. In our malware versus cleanware test – we followed recommendations of [18] in using an equal ratio of malware to cleanware in the training.

The performance of our experiment, malware versus cleanware, shows that all the classifiers tested give comparable results with small variations. RF (Random Forest) gives the best average performance which is 96.9% compared to other classifiers while IB1 shows lower performance, which is 91.34%.

Table 3 also shows the average classification results using meta-classifier AdaBoostM1 on top of base classifiers. The results show that the meta-classifier improves the accuracy for all classifiers tested except IB1, which shows slightly lower performance. Based on these results, the best accuracy rate is 97.3% (AdaBoostM1 with RF) and the lowest accuracy is 92.7 (AdaBoostM1 with

Figure 4 shows our classification methodology. Each malware file is tested with cross-validation once whereas each cleanware file has cross-validation performed on it three times in correspondence with each each

IB1). Figure 5 graphically depicts the comparison of classification accuracy of base classifiers with meta-classifiers. Our results show that classification of cleanware versus malware using behavioural features can be used to achieve high classification accuracy.

Table 3. Average malware versus cleanware classification results

	Base Classifier			Meta Classifier		
	FP	FN	Acc	FP	FN	Acc
SMO	0.01	0.08	0.954	0.0	0.03	0.961
IB1	0.06	0.07	0.9134	0.02	0.06	0.927
DT	0.03	0.05	0.965	0.01	0.04	0.971
RF	0.01	0.06	0.969	0.01	0.04	0.973

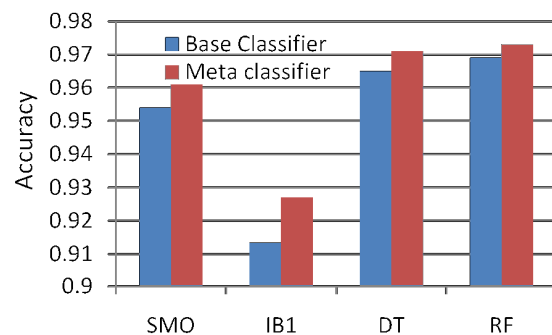


Figure 5. Comparison of average classification accuracy between base and meta classifiers.

5. Malware Family Classification

In our second experiment we achieve further refinement of our classification by comparing malware across and within families. By comparing individual

malware files based on their behaviour, we determine similarities which indicate a commonality. As we shall show, the results of this work, to a large extent, validate the pre-classification within the CA zoo.

This work can also be used in quickly identifying an unknown file by testing it against the known families. If the classification test identifies it as being a member of a family already considered malicious, then the file can be classified as malicious without further analysis.

5.1. Data Preparation for Classification

The file preparation process is similar to that described in section 4.2. The only difference is that first we generate the string list based on entire families and then create a hash table for every family and store all string information along with the module frequency, into our hash table. For example, let $G_i = \{s_1, s_2, s_3, \dots, s_n\}$ be the set of distinct strings of a family F_i , where $s_1, s_2, \dots, s_n$ represents the strings; for instance - s_1 "kernel32", s_2 "advapi32", s_3 "GetModuleHandleA" ..., s_n "OpenProcess".

Each malware family F_i comprises a number of files, and each file is represented here by its module id M . Thus, for any particular family F_i , we can write

$$F_i = \{M_1^i, M_2^i, \dots, M_q^i\} \quad (1)$$

where q is the number of files in the family.

We can also write each M_j^i as a 3-tuple:

$$M_j^i = \langle s, f_s, f_m \rangle \quad (2)$$

to represent each file in this family; here s is the string; f_s is the total number of occurrences of string s within the family and f_m is the number of occurrence of string within a file.

For each malware file, we then test which strings in our family list that it contains. After inserting all the strings along with corresponding values into the hash table, we follow the same process as described in section 3.3.

The process of malware family classification is similar to that described in 4.3. The only difference is in how we construct the training and test data. In our testing, we select a particular family and split them into $K=10$ different folds (as in Figure 5, section 4.3). For training the classifiers, we choose an equal number of positive (from the family) and negative (non-family) files. We construct the negative set by randomly selecting files from the other families. We invoke Weka libraries to train the classifier and evaluate the test data. The same rotating process applies for all other families.

5.2 Experimental Results

The experimental setup and the methodology of our family classification system is similar to that presented in section 3.4. We tested four base classifiers: SVM, Random Forest, Decision Table and IB1, in our experiments. We used the same data set as listed in Table 1. Table 4 shows the accuracy of our family classification for the tested classifiers. It is shown from Table 4 that RF has best performance and IB1 the worst. Some families show over 99% accuracy, such as bambo, looked and agabot, whereas emerleox and clagger show the worst performance. We can deduce that the number of strings within the file impacts on the

classification performance. Our experimental study shows that we achieved an average accuracy of 95.5 % using the RF classifier.

Table 4 also shows the average malware family classification results using the meta-classifier AdaBoostM1 on top of the base classifiers. The results show that the meta-classifier improves the accuracy on all classifiers. Based on these results, the best accuracy rate is 97.4% (AdaBoostM1 with RF). Figure 6 compares the classification accuracy of base classifiers with meta-classifiers.

Table 4: Average family-wise malware classification results

	Base Classifier			Meta Classifier		
	FP	FN	Acc	FP	FN	Acc
SMO	0.033	0.083	0.945	0.032	0.082	0.951
IB1	0.055	0.086	0.931	0.045	0.085	0.939
DT	0.037	0.106	0.925	0.034	0.10	0.931
RF	0.038	0.066	0.957	0.036	0.065	0.958

As the families are not all of the same file size, we therefore calculated a weighted average, where each family was weighted according to the formula $nf_i n_f$, where nf_i is the number of files in family f_i and n_f is the total number of files (across all families). According to the weighted average, the RF classifier achieved the best overall results in our family classification with 97.4% accuracy, as shown in Table 5.

Table 5: Weighted average family-wise malware classification results

	Base Classifier			Meta Classifier		
	FP	FN	Acc	FP	FN	Acc
SMO	0.018	0.086	0.952	0.017	0.084	0.952
IB1	0.062	0.094	0.925	0.06	0.08	0.935
DT	0.029	0.112	0.931	0.02	0.11	0.939
RF	0.021	0.046	0.971	0.021	0.04	0.974

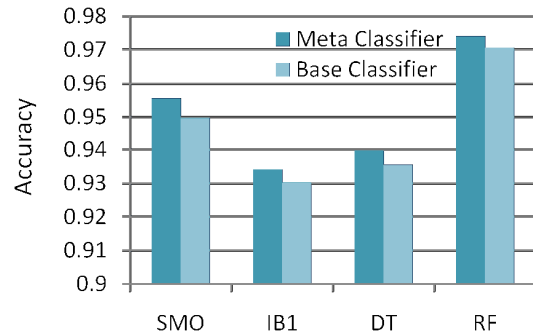


Figure 6. Comparison of family classification results with and without AdaBoostM1

6. Discussions

6.1 Cleanware versus Malware

Distinguishing malware from cleanware is more critical for malware detection than malware family classification, and can offer a first line of defence against malware. Recent research trends are moving toward the classification of clean files from malicious file. [7, 9, 15, 17] We have investigated some recent work classifying malware versus cleanware and compared it with ours. Table 6 summarizes this comparison across results achieving at least 80% accuracy, and compares their outcomes with those of our method.

Santos et.al in [9] used opcode sequence frequencies as features to calculate the cosine similarity between two PE executable files. There are 13189 malware files from 6 malware families and 13000 benign files in their dataset. They show that if they select an appropriate similarity ratio threshold, their method would be a very useful tool to identify malware variants from benign files; they do not mention the classification accuracy. Ye et al. in [17] used static API call sequences as a feature of associative rule learning. They test 35000 malware and 15000 cleanware files and their classification accuracy is 88%. Wang et al. in [15] used static API call sequences from 714 files which are pre-identified as either normal (361) or virus programs (353); they obtain 93.71% classification accuracy. Moskovitch et al. in [7] used the text categorization process and examined the relationship between the MFP (malicious file percentage) in the test set, which represents real-life scenario, and in the training-set, which being used for training the classifier. The found that the best mean performance is associated with a 50% MFP in the training set.

Table 6. Comparison of our model with existing work

	Experimental Data	Method	Accuracy
Santos et.al (2010)[9]	6(F)/13189(m)/13000(c)	Static feature extraction	No accuracy provided.
Ye et.al (2010)[17]	35000(m)/15000(c)	Static feature extraction	88%
Wang et.al (2009)[15]	353(m)/361 (c)	Static feature extraction	93.71%
Moskovitch et al. (2008) [7]	7688(m)/22735(c)	Static feature extraction with MFP	95%
Our method	1369(m)/454(c)	Dynamic feature extraction with MFP	97.3%

Note: F: the number of malware family;
S: the total number of files;
m: the number of malware files;
c: the number of clean files;

6.2 Malware Family Classifications

We also compare our empirical performance of malware family classification with similar existing methods and summarised it in Table 7. We have considered only the recent results in Table 7 which achieved classification accuracy of 80% or above.

Sathyannarayan et al. in [10] used static extraction to extract API calls from known malware in order to construct a signature for an entire class. The API calls of an unclassified file of malware can then be compared with the 'signature' API calls for a family to determine if the file belongs in the family or not. They tested eight families with 126 files in total, but no specific classification accuracy was provided. Ahmed et al. in [1] proposed a composite method which extracts statistical features from both spatial and temporal information available in run-time API calls. There are 516 malware files in their dataset and they obtained 96.3% classification accuracy. Wagener et al. in [14] tested a small set of malware, 104 files, by using dynamic extraction technologies; they did not provide classification accuracy. The authors in [4] traced the behaviour of malware in a virtual machine environment and used these traces to extract string information. They then applied SVM classifiers and overall performance was determined to be approximately 83.3%. Our method of using dynamic API features got accuracy over 97%, which clearly outperforms all other methods mentioned here.

Table 7. Comparison of similar existing techniques with our method

Paper	Exp. Data	Method	Accuracy
Sathyannarayan et.al(2008) [10]	8(F)/126(S)	Static	Did not mention the quantitative accuracy
Tian, R et.al. (2008) [12]	7(F)/721(S)	Static	87%
Tian, R et.al. (2009) [13]	13(F)/1367(S)	Static	97%
Ahmed et.al. (2009) [1]	516(S)	Dynamic	96.3%
Wagener et.al. (2008) [14]	104(S)	Dynamic	Did not mention the quantitative accuracy
Zhao et.al. (2010) [18]	11(F)/3996(S)	Dynamic	83.3%
Our method	1673(S)	Dynamic	97.4%

7. Conclusions and Future Work

In this paper, the behavioural information from both malware and cleanware binary files are extracted to classify software in two different ways: i) malware versus cleanware classification and ii) malware family classification. The extracted API calls form the basis for modelling the behavioural patterns of software in order to distinguish between classes. Our contributions to the field of malware detection based on behavioural feature analysis are fourfold. The first and foremost contribution is the outlining of a methodology to

extract relevant behavioural features of API calls; such features include hooking of the system services and creation or modification of files. The second contribution is the provision of a statistical analysis of the API calls from log files generated by executing the files in a virtual environment. The third contribution is a method of distinguishing malware from cleanware using a 2-class classification model; this model achieves 97.3% performance in terms of accuracy. The fourth contribution is malware family classification, a similar process to that of testing cleanware versus malware, and this model achieves a classification accuracy of 97.4 %.

Our methodology could be used to provide effective defence against zero-day malware, as cloud computing requires real-time, timely responses of the anti-malware analysis system. In future work we will expand our data set and test unknown malware against the known families. We will continue to look for features which lead to an ever better classification, approaching 100%.

References

- [1]. AHMED, F., HAMEED, H., SHAFIQ, M. Z., AND FAROOQ, M. Using spatio-temporal information in api calls with machine learning algorithms for malware detection. In *AISec '09: Proceedings of the 2nd ACM workshop on Security and artificial intelligence*, pages 55–62, New York, NY, USA, 2009. ACM.
- [2]. CANZANESE, R. J., OYER, M., MANCORIDIS, S., AND KAM, M. A survey of reverse engineering tools for the 32-bit Microsoft windows environment.
- [3]. COVER, T. M., AND THOMAS, J. A. Elements of Information Theory. Wiley-Interscience, 99th edition, August 1991.
- [4]. HUNT, G., AND BRUBACHER, D. Detours: binary interception of win32 functions. In *Third USENIX Windows NT Symposium*, page 8. USENIX, 1999.
- [5]. K-fold Cross Validation, Available at :! [http://en.wikipedia.org/wiki/Cross-validation_\(statistics\)](http://en.wikipedia.org/wiki/Cross-validation_(statistics))
- [6]. MOSER, A., KRUEGEL, C., AND KIRDA, E. Limits of static analysis for malware detection. In *Computer Security Applications Conference, 2007. ACSAC 2007. Twenty-Third Annual*, pages 421–430, Dec. 2007.
- [7]. Moskovitch, R., Stopel, D., Feher, C., Nissim, N. and Elovici, Y. Unknown Malcode Detection via Text Categorization and the Imbalance Problem, *ISI 2008*, pp(156-181), 2008.
- [8]. RAVINDRAN, R., AND BROOKS, R. R. Automated reverse engineering tool.! In *10th Colloquium for Information Systems Security Education*, University of Maryland, pp 103-108, 2006.
- [9]. SANTOS, I., BREZO, F., NIEVES, J., PENYA, Y. K., SANZ, B., LAORDEN, C., AND BRINGAS, P. G. Idea: Opcode-sequence-based malware detection. In *Engineering Secure Software and Systems Second International Symposium, ESSoS 2010 Pisa, Italy, February 3-4, 2010 Proceedings*, volume 5965/2010, pages 35–43. Springer Berlin / Heidelberg, 2010.
- [10]. SATHYANARAYAN, V. S., KOHLI, P., AND BRUHADESHWAR, B. Signature Generation and Detection of Malware Families. In *Information Security and Privacy 13th Australasian Conference, ACISP 2008 Wollongong, Australia, July 7-9, 2008 Proceedings*, pages 336–349, Berlin, Heidelberg, 2008. Springer-Verlag.
- [11]. SCHIPKA, M. White Paper: The Online Shadow Economy: A Billion Dollar Market For Malware Authors, Available at: http://www.fstc.org/docs/articles/messagelabs_online_shadow_economy.pdf
- [12]. TIAN, R., BATTEN, L., ISLAM, R., AND VERSTEEG, S. Function Length as a Tool for Malware Classification, *Malware*, 2008.
- [13]. TIAN, R., BATTEN, L., ISLAM, R., AND VERSTEEG, S. An Automated Classification System Based on the Strings of Trojan and Virus Families, *Malware*, 2009.
- [14]. WAGENER, G., STATE, R., AND DULAUNOY, A. Malware Behaviour Analysis. *Journal in Computer Virology*, 4(4):279–287, November 2008.
- [15]. WANG, C., PANG, J., ZHAO, R., AND LIU, X. Using API Sequence and Bayes Algorithm to Detect Suspicious Behavior. *Communication Software and Networks, International Conference on*, 0:544–548, 2009.
- [16]. XU, J.-Y., SUNG, A. H., CHAVEZ, P., AND MUKKAMALA, S. Polymorphic Malicious Executable Scanner by API Sequence Analysis. In *HIS '04: Proceedings of the Fourth International Conference on Hybrid Intelligent Systems*, pages 378–383, Washington, DC, USA, 2004. IEEE Computer Society.
- [17]. YE, Y., LI, T., JIANG, Q., AND WANG, Y. CIMDS: Adapting Postprocessing Techniques of Associative Classification for Malware Detection. *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on*, 40(3):298–307, may 2010.
- [18]. ZHAO, H., XU, M., ZHENG, N., YAO, J., AND HO, Q. Malicious executables classification based on behavioral factor analysis. e-Education, e-Business, e-Management and e-Learning, *International Conference on*, 0:502-506, 2010.